LagLens: Seeing What a Robot Actually Executes When Its Policy Thinks Too Slowly

Theme: VIS + AI · **Live demo:** https://joowan1108.github.io/laglens/ · **Source:** github.com/joowan1108/laglens (`src/`; run locally: `python3 -m http.server 8000 --directory src`)

Modern robot policies (neural networks that map camera images, and for vision-language-action models also a text instruction, to motor commands) predict a **chunk** of future actions from one observation and execute its first *h* actions **open-loop**, i.e. without looking at the scene again (*h* = execution horizon). Because inference takes time *d* (latency), the robot either **waits** (synchronous) or keeps executing an old chunk and switches when a new one arrives (**asynchronous**). Researchers tune the strategy, the latency budget and the execution horizon from aggregate success curves. LagLens replays a policy from the **same 30 initial states** under **14 execution schedules** (plus a noise replicate) on **two tasks**, the hard PushT task and Catch the ball, in which the ball keeps rolling while the robot thinks, logs every predicted chunk, and links four d3.js views so that an analyst can go from "which schedule is robust?" down to "which chunk switch pushed the block away?".

1 · What: Data Abstraction

Source. We generated the data ourselves (no public dataset logs per-inference chunks). In the simulated **PushT** task, a round pusher (the *agent*, seen from above) must push a T-shaped block onto a fixed T-shaped goal. The pretrained `lerobot/diffusion_pusht` Diffusion Policy [2] turns the last 2 camera frames into a chunk of 16 actions (2-D target positions of the agent) by iteratively removing noise from a random start (10 DDIM denoising steps, CPU, 10 Hz: 1 step = 100 ms); since every chunk starts from random noise, outcomes are stochastic. A **schedule** = strategy × latency *d* × execution horizon *h*: **sync** (hold still for *d* steps, then execute *h* actions), **naive async** (keep executing the old chunk while inference runs, switch after *d* steps and skip the *d* outdated actions), and **inpaint async** (as naive, but the new chunk is generated with the already committed actions held fixed and the old plan softly blended in over the overlap; a training-free approximation of Real-Time Chunking [8]). Async schedules with $d = 0$ equal sync $d = 0$. We show $d \in \{0, 2, 4\} \times h \in \{4, 8\}$, i.e. **14 schedules**, and the first 30 seeds (1000–1029, fixed before looking at outcomes); a first, larger grid (46 schedules × 50 seeds) was too dense to read and is kept only as raw logs. A **replicate** of sync *d*0 *h*8 with different sampling noise measures chance flips. **Catch the ball** (ours) is a second task in which the world keeps moving: the same pusher and PD controller (with the same walls) must touch a ball (radius 25 px) that rolls at 42–100 px/s and bounces off the walls, within 80 steps (8 s); speed, direction and positions differ per start state. Its policy is a small Diffusion Policy (MLP denoiser, same 16-action chunks and 2-step observation of pusher and ball, actions relative to the pusher) that we trained in 4 min on CPU from 4,000 scripted interception demonstrations. Ball size and deadline were calibrated on separate seeds (5000–5099) so that sync without latency succeeds in about 90 % at both horizons, before any schedule with latency was run on the evaluation seeds. Both tasks share seeds and schedules.

Dataset type: a multidimensional **table** of episodes (900 items over both tasks × 23 attributes; PushT 47 KB), each linked to two **temporal** tables, steps (131,872 items) and chunks (21,689 items, each with 16 predicted 2-D actions), and to a 2-D **spatial** field (512 × 512 px workspace); 21 MB of JSON lazy-loaded per schedule. **Keys:** task, strategy, latency, horizon, replicate (→ schedule), seed (→ episode), t (→ step), chunk id (→ chunk). Everything else is a value.

Attribute	Type, order	Levels / range	Meaning	Attribute	Type, order	Range	Meaning
strategy KEY	nominal	sync, naive, inpaint	chunk replacement rule	coverage_final · dist_final	ratio ↑ · ↓	0.00–0.99 · 32–426 px	T-block/goal overlap (PushT) · pusher–ball distance (Catch)
latency d KEY	ordinal (ratio)	0<2<4 steps	inference delay	steps_to_success	ratio ↓	74–300 · 12–80 (null if failed)	time to finish
horizon h KEY	ordinal (ratio)	4<8 steps	open-loop actions per chunk	jump_mean / max	ratio (px) ↓	0.3–381.9 / 1–949	jump at a chunk switch (chunk-boundary discontinuity)
replicate KEY	nominal (binary)	0, 1	noise re-run	think_motion_mean / max	ratio (px) ↓	0.0–227.8 / 0–499	robot motion while the policy thinks
seed KEY	nominal	1000–1029 (30)	paired initial state	idle_frac	ratio ↓	0.00–0.60	share of steps without action
success	nominal (binary)	0, 1	PushT: coverage ≥ 95 % in 300 steps; Catch: touch in 80	jerk · path_len · n_inferences · n_switches · inner_step	ratio	1.3–232.9 · 111–6158 · 2–75 · …	smoothness (2nd difference) · distance · # chunks · # hand-overs · step size inside a chunk

Derived metrics. *Chunk-boundary discontinuity* (columns `jump_*`; the position gap at chunk transitions studied in [8,11,12]): at every switch from chunk *k* to *k*+1, the distance between the first action executed from *k*+1 and the action *k* had planned next. *Motion while thinking*: for every executed chunk, the distance the robot moved between the camera picture the chunk was planned from and the step the chunk was ready, i.e. how much the scene changed that the plan did not see. *Staleness* (`staleness_mean`, 1.4–7.5 steps): age of that picture when each action is executed; it averages $d + (h-1)/2$ and is identical for all strategies at equal *d*, *h*, so it does not separate them. Step values: agent x, y and block x, y, θ (PushT) or ball x, y (Catch) (interval, px; θ cyclic), action x, y, chunk id, index j (0–15), age (steps since its observation), held (1 = idle step), coverage or distance.

2 · Who: Personas



Mara Jensen · primary · 28 · PhD student, robot learning lab, Copenhagen

Goals/needs: choose strategy and horizon for a real arm whose GPU server is 60–250 ms away; explain *why* episodes fail, with per-episode evidence aligned with what the policy planned.

Pain points: only success tables and videos; cannot tell whether a failure came from waiting, an outdated plan or a jump; real-robot reruns are expensive.



Arjun Mehta · secondary · 37 · ML platform engineer, robotics start-up, Seoul

Goals/needs: set a latency budget and a default strategy for the async serving stack; know whether a drop is real or a few lucky seeds.

Pain points: papers report curves for other robots; no shared vocabulary with researchers. **Difference:** systems vs. policy expertise; aggregate (T1, T2) vs. episode-level (T3, T4) questions.

Portraits: public-domain StyleGAN images (Wikimedia Commons); the people do not exist.

3 · Why: Task Abstraction

#	Action → target	Why	Expected outcome
T1	**Compare → trends** of success and of each strategy's cost (idle time, motion while thinking, discontinuity) across schedules	pick a schedule for the latency the lab actually has	robust region and a latency budget per strategy
T2	**Locate → outliers**: initial states whose outcome flips between a baseline and a candidate schedule	aggregate rates hide which situations break; paired seeds isolate the schedule effect	a set of fragile seeds; flips separated from chance
T3	**Identify → features** in time and space: where idling, outdated plans and discontinuities occur in a flipped episode	diagnose the failure mechanism, not just its rate	a causal hypothesis tied to specific chunks
T4	**Discover → correlation** between boundary discontinuities and task outcome over paired episodes	test the claim that failures come with larger chunk-boundary artifacts [12]	evidence for or against a strategy, per seed

4 · How: Vis Idiom Design

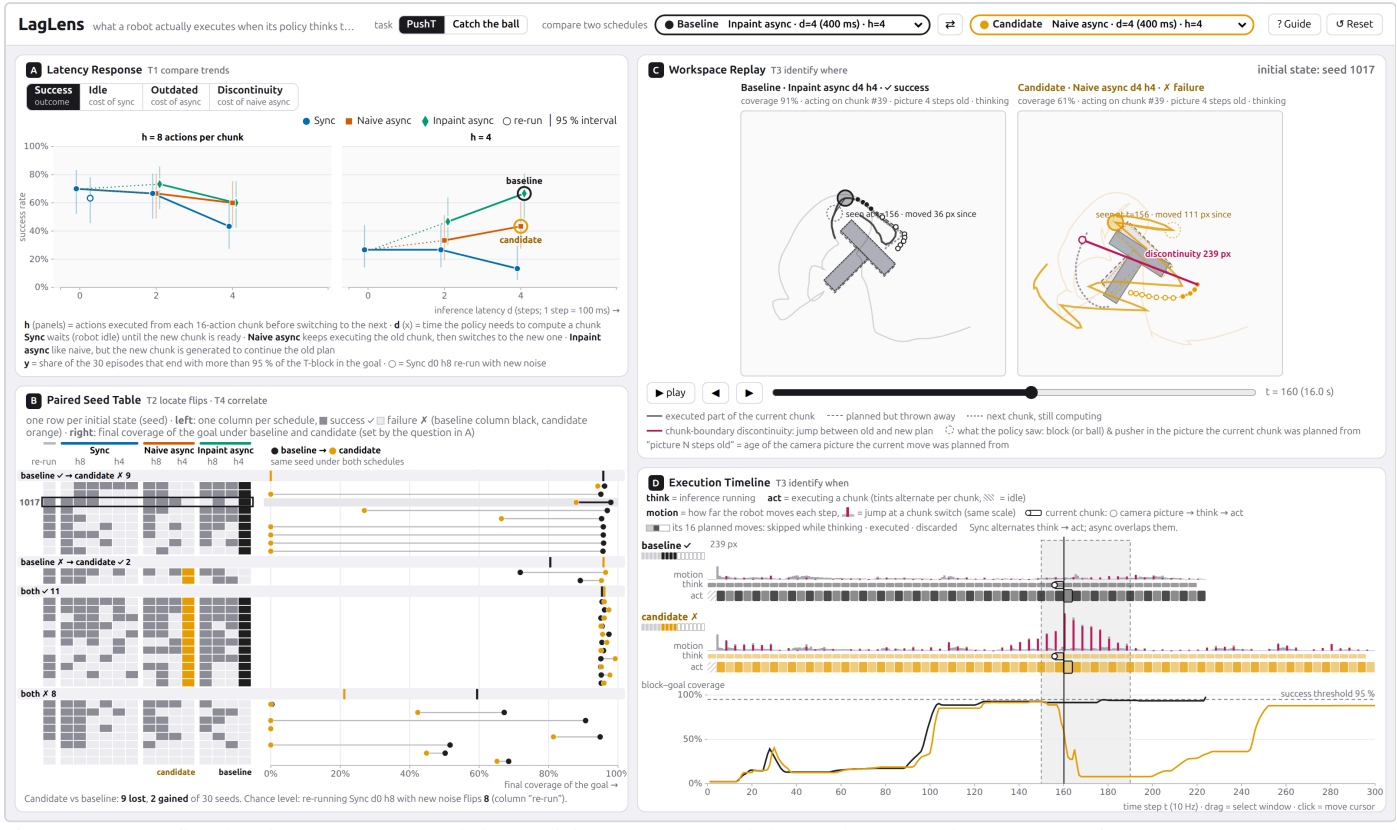


Figure 1. LagLens (live: baseline = inpaint async d4 h4, candidate = naive async d4 h4, seed 1017, window t = 150–190). **A** Latency Response, **B** Paired Seed Table, **C** Workspace Replay, **D** Execution Timeline. All views share one baseline/candidate pair, one seed and one seed selection.

Design goal: make the cost of each strategy visible. Sync pays with **time** (the robot idles while the policy thinks: idle share ≈ $d/(h+d)$, 51 % at $d4$ $h4$), naive async with **discontinuity** (no idling, but the new chunk can pick another *mode*, i.e. another way of pushing: mean discontinuity 40.3 px at $d4$ $h4$ in PushT), both async strategies with **outdated plans** (the robot keeps moving while the policy thinks: inpaint 30.9 px and naive 43.9 px vs. 11.5 px sync at $d4$ $h4$), inpaint async with small discontinuities (7.8 px). Which cost dominates depends on the task, so the header has a **task switch** (PushT · Catch the ball) that reloads all four views. Each cost has one signature: A's "mechanism" metrics, D's think/act tracks (alternating vs. overlapping) and discontinuity bars in the motion track, view C's magenta discontinuity segment. A first version (5 views, ~10 hues, numbers in every heatmap cell) was judged cluttered; we redesigned it with the VIS literature below (full rationale: `docs/design_rationale.md`).

View (task)	Idiom, marks & channels — why	Interaction
A Latency Response (T1)	Line-chart small multiples, one panel per horizon *h* (shared y); x = latency *d* on its true scale, y = metric (*position on a common scale*, the most accurate magnitude channel [16,17]); strategies **superposed** for direct sync-vs-async comparison, hue + redundant shape [18]. Thin 95 % intervals (Wilson / t) instead of bars [19]; hover gives the **paired** difference to the baseline.	click = candidate, shift-click = baseline; **one question bar** (Success · Idle = cost of sync · Outdated = robot motion while thinking (cost of async) · Discontinuity = cost of naive async) sets A's y *and* B's dumbbells; legend hover; **recomputes on the seeds selected in B**.
B Paired Seed Table (T2, T4)	Composite, Table-Lens-like view [20]: one row per seed shared by an **outcome matrix** (filled = success; baseline column black, candidate orange; columns grouped by strategy) and **dumbbells** baseline → candidate of the attribute that belongs to A's question (position, symlog for discontinuities). Rows grouped by baseline→candidate change (*spatial region*), robust seeds first, so flips break their band; a **re-run** column gives the chance level; band ticks = medians.	no controls of its own; click header = candidate; click cell = open episode; **brush** dumbbells or click a band = select seeds (cross-filters A [21], dims rows).
C Workspace Replay (T3)	Juxtaposed baseline | candidate maps at equal scale (true geometry) [18]. Only the current chunk (filled = executed, hollow dashed = thrown away), the chunk still being computed (dotted grey) and the latest discontinuity (magenta segment, length = gap); a dotted outline of **what the policy saw** (block and pusher in the camera picture of the current chunk) with an arrow to the pusher now; status line in strategy terms ("idle: waiting for inference").	play / scrub / keys; hover a planned point = when planned and executed; click = jump; D's window highlights the path.
D Execution Timeline (T3)	Gantt lanes on a shared time axis [22]: per run a **think** track (inference intervals) and an **act** track (chunk spans, alternating tints, hatched = idle); a **motion** track (movement per step as a grey area; discontinuities as magenta bars on the same scale, so a jump is read against normal motion); coverage lines with the 95 % threshold. The chunk at the cursor (or hovered) is traced **picture ○ → think → act**, and a 16-cell strip shows which of its planned moves were skipped while thinking, executed or discarded.	**brush** a window (→ view C), hover a chunk (→ its plan in view C), click = cursor.

Hue has one meaning per view family: baseline near-black, candidate orange (identity); sync blue, naive vermillion, inpaint green (strategy, with ●■◆); magenta = discontinuity; all else grey. CVD-checked with Machado et al.'s simulation (baseline/candidate $\Delta E_{OK} \geq 48$). Following the rules of parsimony and consistency [23], the former flip matrix and seed-shift plot became one view, and our own walkthrough showed that ten controls with technical names (Δ mode, sort orders, an 8-item attribute list, playback speed) were hard to understand, so they were reduced to the baseline/candidate pair and one four-button question bar named after who pays each cost. Every term and encoding is also defined in place (baseline and candidate named in the header; key lines for *h*, *d*, strategies, cells and tracks under each view), so the guide is not needed to decode the views. **Sketch → implementation:** the sketch (Fig. 2) shows the redesigned layout and is implemented as drawn.

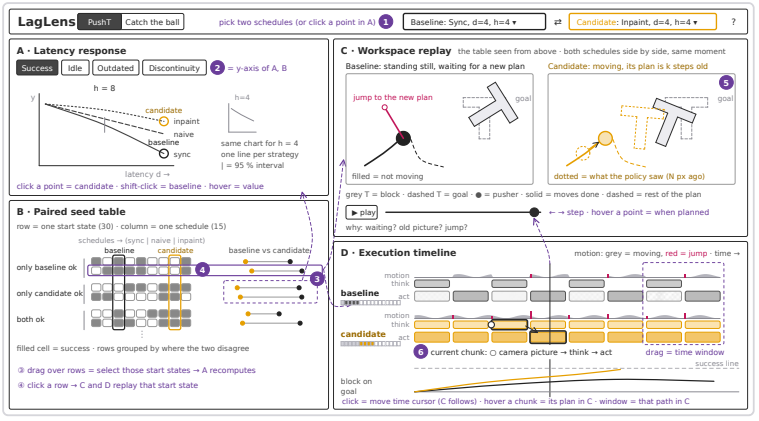


Figure 2. Low-fidelity sketch (purple = interaction and linking; marks are placeholders).

5 · Implementation

- **Interface** (`src/`): plain ES modules + **d3.js v7.9.0 only** (bundled, no other charting library). One shared state object with a `d3.dispatch` bus; each view re-renders only for the state keys it depends on. Per-schedule logs (≤ 5.3 MB) are fetched lazily and indexed once (executed index set per chunk), so scrubbing and hovering stay interactive. URL parameters reproduce any view state (used for every figure here).
- **Data pipeline** (`scripts/`): `generate_rollouts.py` runs all seeds of a schedule in lockstep so that one batched diffusion call serves all environments (CPU, ≈ 2–10 min per schedule); `catch_task.py` / `generate_catch.py` train the Catch policy and run its schedules (seconds); `build_dataset.py` selects the shown grid and derives metrics; `verify_dataset.py` checks the ≥ 100 items / ≥ 6 attributes requirement.
- **Testing**: `ui_test.py` loads the app in headless Chromium, runs 41 interaction checks across all views, and fails on any console error (all pass, 0 errors). Two generator bugs found during analysis (an off-by-one inpainting mask and a dropped chunk when *d* = *h*) were fixed and the affected data regenerated.

6 · Case Study: Mara chooses an execution schedule for 400 ms of latency

Mara's server adds ≈ 400 ms (*d* = 4). She opens LagLens on PushT with sync *d*0 *h*8 as the baseline, the policy's default.

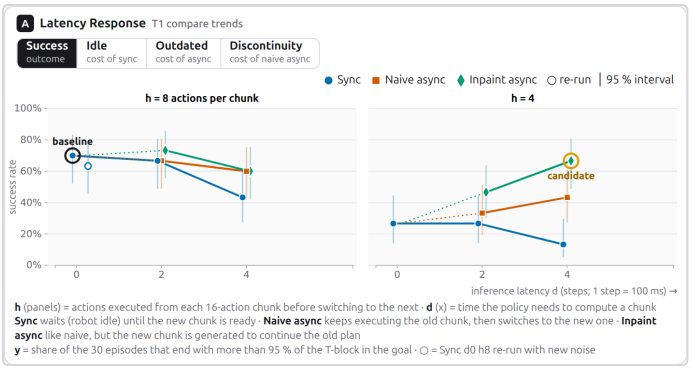


Figure 3. A (PushT): success rate vs. latency per horizon (candidate = inpaint d4 h4).

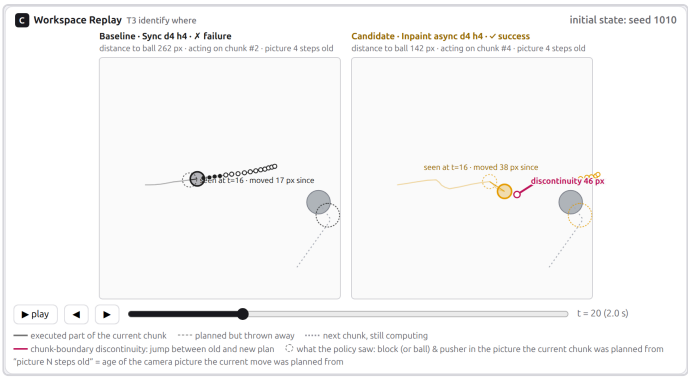


Figure 4. C after the task switch to Catch the ball: seed 1010 at t = 20, baseline = sync d4 h4 (failure), candidate = inpaint d4 h4 (success); dotted circles = where the ball and pusher were in the picture the current chunk was planned from.

Finding 1 (T1): at 400 ms, inpainting is the only strategy that stays flat across horizons. In A (Fig. 3) only inpaint *d*2 *h*8 (73 %) reaches the zero-latency baseline (sync *d*0 *h*8, 70 %). At *d* = 4 the blue sync curves fall in both panels, to 43 / 13 % for *h* = 8 / 4 (idle share up to 51 %), naive async to 60 / 43 %, while inpaint async holds 60 / 67 %. Pressing *Discontinuity* shows why naive degrades at the short horizon: its mean discontinuity grows to 40.3 px at *d*4 *h*4, while inpainting stays ≤ 11.5 px everywhere. The horizon also matters without latency: sync *d*0 succeeds 70 % at *h*8 but only 27 % at *h*4, the jittery re-planning Chi et al. reported [2].

Finding 2 (T1–T3): when the world keeps moving, acting while thinking wins. Switching the task to Catch the ball, A shows sync falling from 90 % without latency to 67 % (*d* = 2) and 47 % (*d* = 4) at both horizons, while async stays at 80–90 % at *d* = 4 (except inpaint *h*8, 67 %). In B (baseline sync *d*4 *h*4, candidate inpaint *d*4 *h*4) 13 seeds are rescued and none lost, against 4 chance flips of the re-run; naive async rescues 14 and loses 3. Seed 1010 (Fig. 4) shows why: sync waits 400 ms before every chunk (idle 51 %), the ball rolls on, and each plan aims where the ball was (dotted circle); at t = 20 sync is still 262 px from the ball, inpaint 142 px, and it touches it at t = 24 like the no-latency run (t = 26), while sync never does. In D, naive async pays with discontinuities: at *d*4 *h*4 its largest jump per episode has a median of 262 px (up to 949 px between planned targets; the walls stop the pusher), because its jumpy motion drives the policy outside its training data, whereas inpainting keeps it ≤ 72 px.

Finding 3 (T2): about one in four PushT outcomes flips by chance, so small differences are noise. Setting the candidate to the **noise replicate** gives 5 losses and 3 gains: 8 of 30 seeds flip with an identical schedule. Naive *d*2 *h*8 and sync *d*2 *h*8 (5 losses / 4 gains each) are therefore indistinguishable from the baseline, whereas sync *d*4 *h*4 (20 / 3) is a real loss. Within each band the most robust seeds come first and the hover reveals fragile ones: seed 1001 succeeds under the baseline but in only 29 % of all schedules, so Mara removes such seeds from her claims.

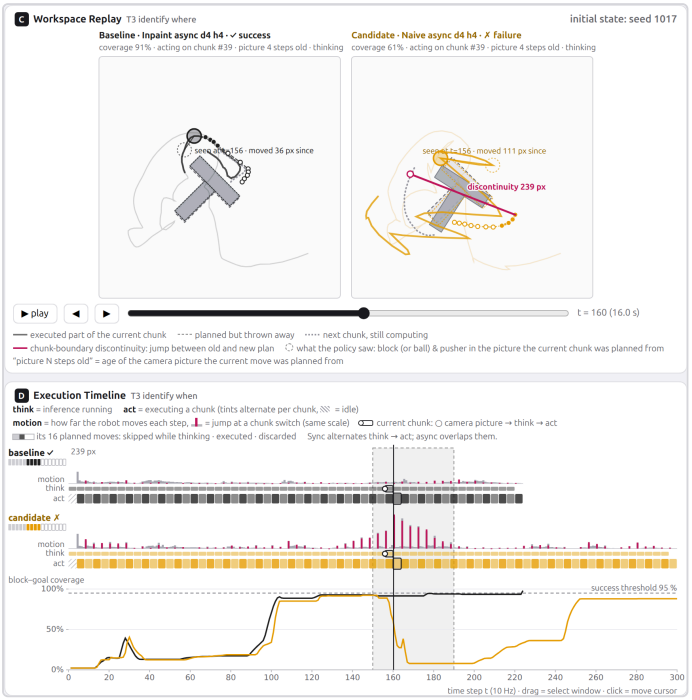


Figure 5. PushT seed 1017, baseline = inpaint d4 h4 (success), candidate = naive d4 h4 (failure); window t = 150–190 brushed in D, cursor at t = 160.

Finding 4 (T3): naive async fails through "mode ping-pong" at chunk discontinuities. For seed 1017 both schedules act on equally old pictures (4–7 steps) and naive async moves somewhat more while thinking (mean 39 vs. 31 px); until t ≈ 150 both hold the block about 92 % in the goal. In D's motion track the candidate's discontinuity bars then grow (55 px at t = 140, 124 px at t = 156) and peak at 239 px at t = 160, while the baseline's stay ≤ 21 px in the window. Scrubbing view C to t = 160 shows the jump: the old chunk planned its next move on the left of the block, the new chunk starts at its right end, and the dotted outline shows that the pusher has moved 111 px since the picture at t = 156 (baseline: 36 px). The pusher swings across the T, which slides out of the goal: coverage falls from 92 % (t = 150) to 61 % (t = 160) and 8 % (t = 168) and recovers only to 93 %, just under the threshold. Inpainting keeps the committed part of the old plan, so the baseline's moves continue and it finishes at t = 224.

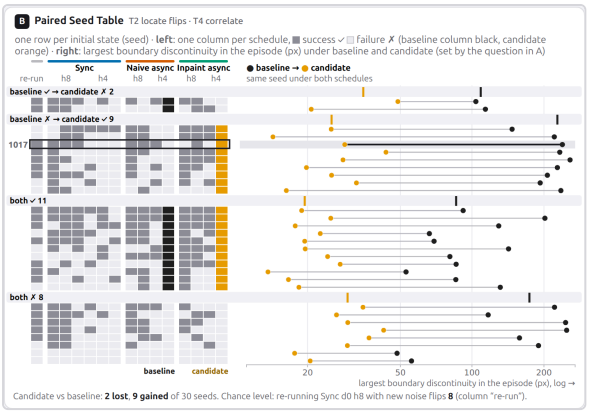


Figure 6. B (PushT) with baseline = naive d4 h4, candidate = inpaint d4 h4, question "Discontinuity": largest discontinuity per episode (log axis); band ticks = medians.

Finding 5 (T4): naive-async failures come with large discontinuities; inpainting shrinks every discontinuity but not every failure. With naive *d*4 *h*4 as baseline and inpaint *d*4 *h*4 as candidate, B (Fig. 6) groups the 30 seeds by outcome change. The median ticks separate the bands: seeds that *failed* under naive async have median max discontinuities of 174–228 px, seeds that succeeded 86–109 px. All 30 dumbbells point left: inpainting lowers the largest discontinuity of *every* seed, to at most 49 px. 9 seeds are rescued and 2 lost (within the noise floor of Finding 3). Yet 8 seeds still fail under inpainting although their discontinuities (median 30 px) are close to those of successful runs (19–25 px), so their failures need another explanation (e.g. reduced reactivity), which Mara inspects next in views C and D; selecting that band re-aggregates A on those 8 seeds. Over all schedules, failed naive episodes have a median max discontinuity of 85 px vs. 58 px for successes, consistent with [12]; for inpainting 28 vs. 24 px.

Outcome. Mara proposes inpaint async with h = 4–8 for $d \approx 4$, reports differences only beyond the 8-flip noise floor, and gives Arjun a latency budget: sync is acceptable only up to d = 2 at h = 8 for PushT, and not at all when objects move (Catch).

7 · Limitations

Two simulated tasks; the Catch policy is small and trained on scripted demonstrations, and the ball ignores contact, so the task isolates the cost of acting late in a moving world rather than modelling a real application; latency is quantised to 100 ms steps; the inpainting strategy is a training-free approximation of RTC; a faster 10-step denoising schedule (DDIM) replaces the original 100-step one (DDPM) for CPU feasibility (the replicate shows that this changes outcomes of individual seeds but not the 63–70 % baseline rate). With 30 seeds, PushT success differences below ≈ 8 flips should not be interpreted.

AI Usage Disclosure

This project was built with **Claude Code (Anthropic, Claude Opus 5.5)**, directed and reviewed by the student. **Written by AI:** the literature survey and topic comparison, all data-generation and preprocessing scripts (including the Catch-the-ball task and its policy training), the complete interface code in `src/` (d3.js), the tests, the proposal slides and sketch, the personas, and this report's text, figures and screenshots (produced by operating the interface through scripted states). **Human:** topic choice among AI-proposed candidates, design requests (more and richer views; later a less cluttered, literature-grounded redesign focused on sync vs. async; a second task that shows the advantage of async; fewer schedules and seeds), deployment approval, review. Numbers in the slides and this report are filled in automatically from the data files. External code, data and images are listed below and in `docs/references.md`; a dated log is in `docs/ai_disclosure.md`.

References & Credits

[1] Zhao et al. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware (ACT). RSS 2023. arXiv:2304.13705
[2] Chi et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. RSS 2023. arXiv:2303.04137
[3] Black et al. π0: A Vision-Language-Action Flow Model. 2024. arXiv:2410.24164
[4] Pertsch et al. FAST: Efficient Action Tokenization for VLA Models. 2025. arXiv:2501.09747
[5] Kim, Finn, Liang. Fine-Tuning VLA Models: Optimizing Speed and Success. 2025. arXiv:2502.19645
[6] Shukor et al. SmolVLA (asynchronous inference). 2025. arXiv:2506.01844
[7] Liu et al. Bidirectional Decoding. ICLR 2025. arXiv:2408.17355
[8] Black, Galliker, Levine. Real-Time Execution of Action Chunking Flow Policies (RTC). NeurIPS 2025. arXiv:2506.07339
[9] Training-Time Action Conditioning for Efficient Real-Time Chunking. 2025. arXiv:2512.05964
[10] VLASH: Real-Time VLAs via Future-State-Aware Asynchronous Inference. 2025. arXiv:2512.01031
[11] SEAM: Smooth Execution of Action-Chunked Motion for VLA Policies. 2026. arXiv:2607.04609
[12] Noise-Space Attribution and Control of Chunk-Boundary Artifact. 2026. arXiv:2603.11642
[13] Lugmayr et al. RePaint. CVPR 2022 · Song et al. DDIM. ICLR 2021.
[14] Munzner. Visualization Analysis and Design. CRC, 2014 · Okabe & Ito colour palette.
[15] Related VIS: DQNViz (VAST 2018), DRLViz (EuroVis 2020), PolicyExplainer (arXiv:2104.02818).
[16] Cleveland, McGill. Graphical perception. JASA 1984 · [17] Heer, Bostock. Crowdsourcing graphical perception. CHI 2010.
[18] Gleicher et al. Visual comparison for information visualization. Inf. Vis. 2011 · Gleicher. Considerations for visualizing comparison. TVCG 2018.
[19] Correll, Gleicher. Error bars considered harmful. TVCG 2014 (VIS).
[20] Rao, Card. The Table Lens. CHI 1994 · Javed, Elmqvist. Composite visualization. PacificVis 2012.
[21] Weaver. Cross-filtered views for multidimensional visual analysis. TVCG 2010.
[22] Plaisant et al. LifeLines. CHI 1996 · Brehmer et al. Timelines revisited. TVCG 2017.
[23] Wang Baldonado et al. Guidelines for using multiple views. AVI 2000 · Machado et al. CVD simulation. TVCG 2009.
Software: d3.js 7.9.0 (ISC) · LeRobot 0.6.1 and the `lerobot/diffusion_pusht` checkpoint (Apache-2.0) · gym-pusht (MIT) · PyTorch (BSD) · diffusers (Apache-2.0) · Playwright (Apache-2.0).